



Journal des Modifications & Architecture Technique

Synthèse exhaustive des résolutions de bugs, fiabilisation des prompts IA, règles de sécurité budgétaire, synchronisation du planning et automatisation des appels d'offres.

Projet : Wedding AI Builder (`wedding-ai-builder`)

Dernière mise à jour : 11 Septembre 2026

Statut global : ✓ Code validé (`tsc --noEmit` : 0 erreur)

Ce document est mis à jour à chaque modification apportée au codebase.

Sommaire des Interventions

1. Correction de la Frise Chronologique & Tri Temporel
2. Refonte du Service Email SMTP (Universel & Haute Performance)
3. Documentation Complète des Variables d'Environnement (.env.example)
4. Sécurisation Métier : Validation des Dates Futures & Montants Positifs
5. Planning Adaptatif (Anti-dates Passées) & Bouton de Régénération IA
6. Clarification du Pense-bête Administratif & Audit Budgétaire
7. Fiabilisation & Déterminisme des Prompts IA (Timeline & Budget)
8. Synthèse des Mécanismes de Sécurité Budgétaire
9. Proposition d'Évolution : Refonte de la Gestion Budgétaire
10. Correction du Planning : Déduplication, Étalement Temporel & Bornage des Délais
11. Synchronisation Temporelle Frise/Planning & Ergonomie Calendrier
12. Choix d'une Date Précise dans le Formulaire « Nouvelle Étape » (Mode Hybride)
13. Moteur de Planification IA avec Dates Précises par Tâche & Groupement d'Activités
14. Automatisation des Appels d'Offres : Pré-sélection du Type de Prestataire & Estimation Budgétaire
15. Enrichissement Automatique des Exigences Spécifiques & Priorités depuis le Quiz
16. Synchronisation & Restitution Intégrale du Profil Prestataire (Site Web, Téléphone, Email & Validation)
17. Importation Sécurisée des Avis Google Business Profile avec Verrou Anti-Usurpation
18. Correction & Support Universel des Vidéos Portfolio (YouTube, Shorts, Vimeo, MP4)

19. Proposition d'Évolution : Affichage du Budget Dédié par Prestation et Saisie du Tarif Chiffré
20. Correctifs du Workflow des Appels d'Offres : Résolution « Projet introuvable », Statuts Dynamiques & Célébration
21. Proposition d'Évolution : Synchronisation Collaborative du Calendrier et des Tâches Préparatoires
22. Système d'Indicateurs et Pastilles de Notification Visuelles sur les Menus de Navigation

1. Correction de la Frise Chronologique & Tri Temporel

Problème identifié

- **Inversion temporelle (voyage dans le temps)** : Les jalons de la frise s'affichaient dans un ordre incohérent (ex: Étape 01 en février 2026, Étape 02 en décembre 2026, Étape 03 en novembre 2026, Étape 04 en août 2026).
- **Syndrome du jour "28"** : 100% des jalons tombaient le 28 du mois parce que le calcul soustrayait des mois entiers sur le jour fixe de la date du mariage sans étalement naturel.
- **Hallucination des dates par l'IA** : Le prompt demandait à l'IA de fournir un champ `idealDeadline` en date ISO. L'IA confondait le numéro de mois calendaire avec `monthsBeforeWedding` et renvoyait des dates incohérentes qui écrasaient le calcul du composant.
- **Absence de tri sur la date finale** : Les éléments étaient affichés sans tri sur `targetDate.getTime()`.

Fichiers modifiés & Détails des changements

1. `app/(couple)/espace-couple/result/page.tsx`

- **Calcul robuste de la date** (`computeMilestoneTargetDate`) : Calcule la date exacte de chaque jalon à partir de `weddingDate` et de son délai `monthsBeforeWedding` (en jours réels avec `30.4375` j/mois).
- **Étalement dynamique des dates** : Application d'un décalage naturel de -4 à +4 jours selon la position du jalon pour éviter la répétition artificielle du jour du mariage sur toutes les étapes.
- **Tri chronologique croissant strict garanti** : Application d'un `sort((a, b) => a.targetDate.getTime() - b.targetDate.getTime())`. L'étape 01 est toujours la plus ancienne, l'étape 02 lui succède dans le temps, jusqu'au Jour J en étape finale.

2. `lib/report/reportHelpers.ts`

- **Mise à jour de** `normalizeMilestones` : Ne se laisse plus court-circuiter par une chaîne `idealDeadline` erratique.
- **Tri chronologique croissant** appliqué sur toutes les étapes retournées pour les dashboards et vues synthétiques.

3. `lib/ai/prompts/timeline.prompt.ts`

- **Contraintes strictes d'ordre chronologique** : Ajout de la règle exigeant un `monthsBeforeWedding` strictement décroissant (12, 10, 9, 8, 6, 5, 4, 3, 2, 1, 0.25, 0).
- **Enchaînement canonique explicité** : Fourniture de la séquence logique standard de l'industrie du mariage (Lieu
→ Traiteur → Photo → Tenues/DJ → Déco → Faire-part → Alliances/Dégustation →
Logistique → Jour J).

- **Conservation de tous les attributs d'analyse** : Maintien et clarification des instructions pour `priority`, `urgency`, `timeNeeded`, `consequences`, `dependencies`, `tasks`, et la couverture exhaustive des préparatifs.

2. Refonte du Service Email SMTP (Universel & Haute Performance)

 Objectif

Rendre le moteur d'envoi d'emails universel et opérationnel en production avec n'importe quel fournisseur SMTP (Brevo/Sendinblue, Mailjet, Postmark, AWS SES, OVH, Gmail, etc.) ou en développement local avec Maildev/Mailtrap.

Fichiers modifiés & Détails des changements

lib/email/send.ts

- **Détection TLS/SSL automatique intelligente :**
 - Port `465` → SSL direct (`secure: true`).
 - Port `587` , `25` ou `1025` → STARTTLS moderne (`secure: false`).
 - Possibilité de surcharge manuelle via `SMTP_SECURE="true"` ou `"false"` .
- **Suppression du blocage** `ignoreTLS` : Le code précédent forçait `ignoreTLS: true` en environnement de développement, ce qui provoquait le rejet immédiat de la connexion par les serveurs SMTP professionnels.
- **Pool de connexions & Performance :** `pool: true` avec `maxConnections: 5` et `maxMessages: 100` en production pour réutiliser les sockets réseau et accélérer l'envoi d'emails.
- **Timeouts réseau stricts :** 10s pour la connexion, 5s pour le handshake de bienvenue, 15s pour le socket (évite le blocage indéfini des workers Next.js en cas d'incident réseau).
- **Formatage d'expéditeur conforme RFC 5322 :** `"<math xmlns="http://www.w3.org/1998/Math/MathML"><semantics><mrow><mrow><mi>F</mi><mi>R</mi><mi>O</mi><msub><mi>M</mi><mi>N</mi></msub><mi>A</mi><mi>M</mi><mi>E</mi></mrow><mi mathvariant="normal">"</mi><mo><</mo></mrow><annotation encoding="application/x-tex">{FROM_NAME}"<</annotation></semantics></math>FROMNAM</math></code>`

`<span class="mord mathnormal" style="margin-right:0.0576em;">E</span></span><span class="mord">"`
`</span><span class="mspace" style="margin-right:0.2778em;"></span><span class="mrel"><</span></span>`
`</span></span></span>{FROM_EMAIL}>` avec nettoyage des guillemets pour une délivrabilité maximale (anti-spam).

- **Logs et diagnostics d'erreur détaillés** : Affichage clair du code d'erreur (ex: `EAUTH`, `ECONNREFUSED`, `ETIMEDOUT`) et du retour serveur en cas d'échec.

3. Documentation Complète des Variables d'Environnement (.env.example)

Objectif

Fournir un template propre, exhaustif et sécurisé pour faciliter le déploiement et la configuration locale ou de production sans divulguer de secrets.

Fichiers créés & modifiés

1. `c:\Users\sidia\OneDrive\Documents\App\wedding-ai-builder\.env.example` **(Création)**

- Documentation de l'intégralité des 10 blocs de configuration :
 1. Application & Sécurité (`NEXT_PUBLIC_APP_URL`, `JWT_SECRET`, `ADMIN_PASSWORD`, `USE_LOCAL_DB`)
 2. Intelligence Artificielle (`OPENAI_API_KEY`, `OPENAI_MODEL`)
 3. Firebase Client Web (`NEXT_PUBLIC_FIREBASE_*`)
 4. Firebase Admin Serveur (`FIREBASE_PROJECT_ID`, `FIREBASE_CLIENT_EMAIL`, `FIREBASE_PRIVATE_KEY`)
 5. SMTP Universel (Blocs prêts à l'emploi pour Maildev, Brevo, Gmail, Mailjet, OVH)
 6. Stripe & Monétisation (`STRIPE_SECRET_KEY`, `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`, ID des tarifs d'abonnements)
 7. Médias Cloudinary (`NEXT_PUBLIC_CLOUDINARY_*`)
 8. Cache Redis (`REDIS_URL`)
 9. Analytics PostHog (`NEXT_PUBLIC_POSTHOG_*`)
 10. Supabase Auth (`NEXT_PUBLIC_SUPABASE_*`)

2. `.env.local`

- Clarification de la section SMTP avec des exemples commentés prêts à décommenter.

4. Sécurisation Métier : Validation des Dates Futures & Montants Positifs

Objectif

Empêcher la saisie de dates passées (mariages, disponibilités prestataires, appels d'offres) et bloquer les valeurs négatives sur les montants financiers et quantités.

Fichiers modifiés & Détails des changements

Côté Frontend (UI & Formulaires)

- `components/quiz/QuestionDate.tsx` : `min={today}`, validation `date >= today` ou `"not-fixed"`, message d'erreur clair et désactivation du bouton Suivant si date passée.
- `components/quiz/QuestionBudget.tsx` : `min={1}`, blocage des touches `-` et `e` à la frappe, validation stricte `> 0`.
- `components/quiz/QuestionGuests.tsx` : `min={1}` pour les adultes, `min={0}` pour les enfants, blocage des touches `-` et `e`.
- `app/(couple)/espace-couple/mariage/page.tsx` : `weddingDate` bloqué au futur (`min={today}`), budget et invités avec `min={0}` et assainissement `Math.max(0)`.
- `components/couple/TenderFormModal.tsx` : `budgetMin` et `budgetMax` `≥ 0` avec vérification `budgetMax >= budgetMin`.
- `app/(couple)/espace-couple/prestataires/nouveau/page.tsx` : Budgets d'appels d'offres avec `min={0}`.
- `app/(couple)/espace-couple/budget/page.tsx` : Dépenses prévues et réelles avec `min={0}`, `step="0.01"` et assainissement dans `saveExpense`.
- `app/(couple)/espace-couple/liste-souhaits/page.tsx` : Prix unitaire `≥ 0`, quantité `≥ 1`.
- `app/wishlist/[shareToken]/page.tsx` : Montant de contribution cagnotte invité `min={1}`, `step="0.01"`.
- `app/(vendor)/espace-prestataire/calendrier/page.tsx` : Blocage de dates d'indisponibilité `min={today}` (interdiction de bloquer des dates passées).
- `app/(vendor)/espace-prestataire/profil/page.tsx` : Tarifs min/max, rayon d'action et années d'expérience avec `min={0}` et `Math.max(0)`.
- `app/(vendor)/espace-prestataire/cadeaux/page.tsx` : Prix de cadeaux avec `min={0}`, quantité `≥ 1`.
- `app/(marketing)/devenir-professionnel/page.tsx` : Formulaire d'inscription pro avec `min={0}` sur tous les champs numériques.
- `app/admin/cagnottes/page.tsx` : Montant de reversement virement `≥ 0.01`.
- `app/admin/parametres/page.tsx` : Prix des forfaits et mois d'engagement `min={1}`.
- `app/admin/abonnements/page.tsx` : Jours d'essai et de report `min={1}`.

Côté Backend (Schémas Zod & Routes API)

- `app/api/vendor/calendar/route.ts` : Rejet systématique de toute date antérieure à aujourd'hui (`>= todayStr`).

- `app/api/couple/project/route.ts` : Refine Zod interdisant une `weddingDate` passée, `guestCount.min(0)`, `budget.amount.min(0)`.
- `app/api/quiz/answer/route.ts` : Date de mariage passée rejetée, budget et invités validés non-négatifs.
- `app/api/couple/tenders/route.ts` : `budgetRange.min >= 0`, `max >= min`, `weddingDate >= todayStr`.
- `app/api/wishlist/items/route.ts` : `price.min(0)`, `quantity.min(1)`.
- `app/api/vendor/proposals/route.ts` : Montant de devis validé `z.number().nonnegative()`.

5. Planning Adaptatif (Anti-dates Passées) & Bouton de Régénération IA

Problème identifié

- **Dates passées sur la frise** : Si un mariage est prévu en mars 2027 et qu'on est en septembre 2026 (délai de 6 mois), l'application appliquait aveuglément un rétroplanning standard de 12 mois (`M-12`), ce qui faisait apparaître des étapes en **mars 2026** (6 mois dans le passé !).
- **Régénération difficile** : L'utilisateur n'avait pas de moyen simple de redéclencher un calcul IA frais sans recommencer le quiz.
- **Visualisation de la réponse brute** : La réponse complète de l'IA n'était pas exportée dans un fichier facilement consultable.

Fichiers modifiés & Détails des changements

1. `app/(couple)/espace-couple/result/page.tsx`

- **Planning adaptatif (compression proportionnelle)** :
 - Détection du nombre de jours réels restants entre aujourd'hui et la date du mariage.
 - Si le délai restant est inférieur au délai standard (ex: 6 mois restants), l'échéancier est compressé proportionnellement sur l'intervalle `[aujourd'hui, Jour J]`.
 - La première étape commence **maintenant (septembre 2026)** et la dernière est le Jour J.
 - Sécurité mathématique absolue : `target < today` est impossible (verrouillé par `safeDaysBefore` et garde-fou).

2. `lib/report/reportHelpers.ts`

- Application de la même logique de planning adaptatif dans `normalizeMilestones` pour éviter toute date dans le passé sur l'ensemble des vues de reporting.

3. `lib/ai/prompts/timeline.prompt.ts`

- Injection du délai réel restant calculé (`monthsRemaining`) dans le prompt envoyé à l'IA.
- Règle stricte imposant à l'IA de ne jamais dépasser ce délai restant pour le premier jalon.

4. `app/api/couple/result/route.ts`

- **Sauvegarde et export automatique :**
 - **Export automatique de 100% de la réponse IA** dans le fichier `data/dernier_plan_ia.json` pour inspection directe.

6. Clarification du Pense-bête Administratif & Audit Budgétaire

Problème identifié

- **Jargon technique incompréhensible pour les futurs mariés :** Les badges du pense-bête affichaient des codes tels que `M-3 À M-1` ou `M-2`. Les utilisateurs ne comprenaient pas la signification ("mois moins 3").
- **Question sur les prix de la bannière Matching :** Les cartes affichant `Traiteur : 2 300 €` et `Photo : 890 €` laissaient planer le doute sur leur provenance (IA ou hasard).

Fichiers modifiés & Détails des changements

1. `app/(couple)/espace-couple/result/page.tsx`

- **Remplacement du jargon par un français naturel** dans `ADMINISTRATIVE_STEPS` :
 - `M-3 à M-1` → « 3 à 1 mois avant le jour J »
 - `M-2` → « 2 mois avant le jour J »
 - `M-3` → « 3 mois avant le jour J »
 - `Avant le mariage` → « 3 à 6 mois avant le jour J »
 - `Dès que possible` → « Dès fixation de la date »
 - `Dès signature des gros contrats` → « Dès les premiers acomptes »
- **Clarification des cartes de matching :**
 - Ces prix sont des valeurs d'illustration statiques (mockup design UI) présentes dans le template promotionnel CTA et non des calculs IA dynamiques.

2. `app/(couple)/espace-couple/planning/page.tsx`

- **Harmonisation complète du calendrier & du sélecteur d'échéances :**
 - Élimination des libellés cryptiques `M-12`, `M-1` dans le menu déroulant de création d'étape → affichage en français clair : « 12 mois avant le jour J », « Jour J (Le grand jour) ».
 - Remplacement du fallback des cartes jalons de la sidebar (`M-<math xmlns="http://www.w3.org/1998/Math/MathML"><semantics><mrow><mrow><mi>m</mi><mi>o</mi><mi>n</mi><mi>t</mi><mi>h</mi></mrow><mi mathvariant="normal">'</mi><mo stretchy="false"></mo><mi>p</mi><mi>a</mi><mi>r</mi><mo>*</mo><mo>*</mo><mtext><</mtext></mrow><annotation encoding="application/x-tex">{month}</annotation>` ) par `\text{«}month')par * *«{month} mois avant` ».

3. Audit & Validation de la Planification Budgétaire

- **Cohérence mathématique stricte :**

- La somme de tous les postes de dépenses est strictement verrouillée : $\sum \text{breakdown} = \text{totalBudget}$ (précision à l'euro près, sans perte d'arrondi).
- La somme de l'ensemble des quotes-parts en pourcentage égale exactement 100%.

- **Référentiel & Ratios sectoriels certifiés (Marché Français) :**

- **Lieu de réception + Traiteur & Boissons** : 45% à 55% du budget total (les deux postes majeurs incompressibles).
- **Photographe & Vidéaste** : 8% à 12% du budget (prestation qualitative professionnelle).
- **Tenues des mariés, Alliances & Beauté** : 8% à 12% du total.
- **Décoration & Fleuriste** : 7% à 11% du total.
- **Musique & Animation / DJ** : 4% à 6% du total.
- **Imprévus & Sécurité (contingency)** : **8% à 12% obligatoire** (standard de gestion des risques wedding planner pour faire face aux surcoûts de dernière minute).
- **Papeterie, Transport, Logistique & Cadeaux** : 4% à 8%.

- **Pondération contextuelle basée sur les réponses du Quiz :**

- **Enfants présents** : Ajustement du ratio traiteur pour tenir compte des menus enfants (coût réduit par rapport aux adultes).
- **Invités venant de loin** : Augmentation automatique du poste hébergement / logistique (+50% d'allocation).
- **Prestataires non retenus** : Si le couple retire un poste (ex: pas de vidéaste ou pas d'officiant), la quote-part est mise à zéro et redistribuée équitablement sur les postes prioritaires sans modifier le budget global.
- **Géolocalisation** : Prise en compte de la tension de marché (Paris / Île-de-France et Côte d'Azur ajustés à la hausse par rapport aux zones rurales).

7. Fiabilisation & Déterminisme des Prompts IA (Timeline & Budget)

Objectif

Éliminer toute ambiguïté dans les consignes transmises au LLM pour garantir que les réponses soient mathématiquement exactes, alignées avec les schémas Zod et conformes aux réalités économiques de l'industrie du mariage.

Fichiers modifiés & Détails des changements

1. `lib/ai/prompts/budget.prompt.ts`

- **Verrouillage mathématique explicite :**

- La consigne stipule formellement que la somme de `breakdown` doit égaier au centime/euro près le `totalBudget`.
- La somme des `percentages` doit être exactement égale à 100%.
- La réserve `contingency` est encadrée contractuellement entre 8% et 12% du total.
- **Réalisme économique sectoriel :**
 - Intégration des ratios réels du marché français : le couple Lieu + Traiteur représente 45% à 60% du budget global.
 - Ajustement automatique selon la zone géographique (+20% à +35% pour Paris/Île-de-France et Côte d'Azur).
- **Fallbacks robustes :** Si le couple omet de renseigner un budget ou une ville, des valeurs par défaut saines (20 000 €, 60 invités, Paris) évitent l'envoi de variables `undefined` au modèle.

2. `lib/ai/prompts/timeline.prompt.ts`

- **Bornage strict du nombre de jalons :** Exigence explicite de 8 à 14 milestones (recommandé 10 à 12) pour respecter le validateur Zod.
- **Typage strict de `nextCriticalStep` :** `daysLeft` doit impérativement être un nombre entier positif de jours.
- **Interdiction formelle des dates passées :** Injection dynamique du nombre de mois réels restant jusqu'au Jour J (`monthsRemaining`).

8. Synthèse des Mécanismes de Sécurité Budgétaire

MÉCANISME	RÈGLE APPLIQUÉE	GARANT / VALIDATEUR
Somme des dépenses	<code>Somme(breakdown) == totalBudget</code>	Schéma Zod + Prompt IA + Reliquat d'arrondi réinjecté
Somme des pourcentages	<code>Somme(percentages) == 100%</code>	Normalisation mathématique (<code>ratios / sum</code>)
Poste Imprévu	8% à 12% du budget global	Obligatoire dans Zod (<code>min 8, max 12</code>) et dans le Prompt
Paliers de risque par catégorie	<code>planned</code> , <code>recommended</code> , <code>realisticMin</code> , <code>realisticMax</code>	Ratios certifiés du marché du mariage français
Dépassements & Économies	<code>overrunEstimate</code> et <code>savingsPotential</code> calculés par poste	Moteur d'audit budgétaire

9. Proposition d'Évolution : Refonte de la Gestion Budgétaire (Enveloppes Cibles IA vs Dépenses Réelles & Acomptes)

✦ Contexte & Diagnostic du problème identifié

Actuellement, lorsqu'un utilisateur clique sur « **Utiliser ce plan** » dans l'espace budget, le système génère des lignes de dépenses génériques brutes (`WeddingExpense`) pour chaque poste (ex : Libellé : `"Traiteur"` , `plannedAmount` : `5 000 €` , `actualAmount` : `null`).

Le cas d'usage réel qui pose problème :

1. Le plan IA prévoyait par exemple **5 000 €** pour le traiteur.
2. Le couple signe un contrat traiteur à **5 000 €** et verse un **acompte de 1 500 €** (avec 3 500 € à régler plus tard).
3. L'utilisateur clique sur « *Nouvelle dépense* » et enregistre : Libellé `"Acompte traiteur"` , Prévu `5 000 €` , Réel `1 500 €` .
4. **Effet indésirable constaté** : Le montant planifié de la catégorie devient **10 000 €** (`5 000 € (IA) + 5 000 € (nouvelle dépense)`), doublant artificiellement le prévisionnel. De plus, la notion d'acompte et de solde restant à payer n'est pas formalisée.

💡 Spécification de la Solution Proposée

1. Découplage clair : « Enveloppe Budgétaire Cible » vs « Dépenses Réelles »

- **L'Enveloppe Budgétaire** (issue du plan IA ou ajustée manuellement) représente l'**objectif / le plafond cible** alloué à la catégorie (ex : 5 000 € pour le traiteur). Elle ne doit pas être enregistrée comme une simple dépense dans la liste.
- **Les Dépenses Réelles** représentent les contrats, acomptes et factures effectivement engagés auprès des prestataires.

2. Enrichissement du formulaire de dépense

Pour chaque dépense ou contrat saisi par le couple :

- **Montant total du devis / contrat** (ex : 4 800 €).
- **Acompte / Montant déjà payé** (ex : 1 500 €).
- **Reste à payer (Solde)** calculé automatiquement (`4 800 - 1 500 = 3 300 €`).
- **Statut de paiement** : *Acompte versé*, *Payé en totalité*, ou *En attente de règlement*.

3. Algorithme de calcul intelligent par catégorie

Pour chaque catégorie (ex : Traiteur) :

- **Budget Cible (Plan IA)** : `5 000 €`
- **Montant Engagé (Devis signés)** : `4 800 €` → *Marge / Économie prévisionnelle* : `+200 €`
- **Déjà Payé (Acomptes versés)** : `1 500 €`
- **Reste à payer (Trésorerie future)** : `3 300 €`

Règle d'absorption anti-doublon :

- Tant qu'aucune dépense réelle n'est saisie pour une catégorie, le montant planifié affiché est celui de l'**Enveloppe Cible IA**.
- Dès qu'au moins une dépense réelle est créée dans cette catégorie, ce sont les **contrats réels** qui alimentent le prévisionnel, supprimant tout risque de double comptage avec l'estimation IA initiale.

10. Correction du Planning : Déduplication, Étalement Temporel & Bornage des Délais

Problèmes identifiés

1. **Doublons massifs des tâches (72 au lieu de 36)** : Une boucle d'auto-import intempestive côté client déclenchait 36 requêtes `POST` à chaque chargement de `/espace-couple/planning`, dupliquant chaque tâche dans la base.
2. **Entassement artificiel sur le 1er du mois** : Toutes les tâches d'un mois étaient brutalement affectées au 1er jour (`target.setDate(1)`), saturant le 1er du mois (6+ tâches) et laissant le reste du mois complètement vide.
3. **Absence de contexte thématique** : Les tâches s'affichaient sans mention de leur Jalon d'origine (*Planifier la papeterie, DJ & Animation*), donnant une impression d'actions aléatoires.
4. **Sélecteur de mois non borné dans la modal** : Le formulaire d'ajout d'étape affichait les mois jusqu'à 24 mois avant le mariage, même si le mariage a lieu dans 6 mois (permettant la sélection de dates passées).

Fichiers modifiés & Détails des changements

1. `app/api/couple/tasks/route.ts`

- **Déduplication & assainissement automatique** :
 - Filtrage strict dans `GET` par clé unique `title + monthsBeforeWedding`.
 - Nettoyage et suppression automatique des doublons résiduels en base de données pour ramener immédiatement le projet aux 36 tâches uniques réelles.

2. `app/(couple)/espace-couple/planning/page.tsx`

- **Suppression de l'auto-import en boucle** : L'importation initiale n'a lieu que si la collection de tâches est rigoureusement vide (`existingTasks.length === 0`).
- **Étalement réaliste au fil du mois** :
 - Remplacement de `setDate(1)` par une répartition intelligente des tâches par semaine (ex : jour 7 pour la 1ère tâche, jour 15 pour la 2ème, jour 23 pour la 3ème).
 - Pour les tâches du Jour J (`monthsBeforeWedding <= 0`), la date cible est fixée exactement le jour du mariage.
- **Bornage strict du sélecteur de mois dans la modal** :
 - Le champ « Mois avant le mariage » calcule dynamiquement `maxSelectableMonths` à partir de la date réelle du mariage.

- Si le mariage est dans 6 mois, les options s'arrêtent strictement à « **6 mois avant le jour J** » (impossible de choisir 12 ou 24 mois dans le passé).
- **Association des Jalons thématiques :**
 - Ajout d'un badge contextuel (ex : 📅 *Planifier la papeterie*, 🎵 *Réserver le DJ & l'animation*) dans la popup des détails du jour pour chaque tâche.

11. Synchronisation Temporelle Frise/Planning & Ergonomie Calendrier

🎯 Problèmes identifiés

1. **Écart de dates entre la Frise (/result) et le Planning (/planning) :**
 - Sur la frise, les dates étaient calculées par un algorithme adaptatif compressé évitant les dates passées.
 - Sur le calendrier, les dates étaient calculées indépendamment avec un décalage mensuel brut (`weddingDate - monthsBeforeWedding`), créant un décalage flagrant où une étape n'apparaissait pas au même mois entre la frise et le calendrier.
2. **Ergonomie dégradée du calendrier :** Pour voir les tâches d'une journée, l'utilisateur devait obligatoirement cliquer avec précision sur le minuscule lien `+3 autres`.

🔧 Fichiers modifiés & Détails des changements

1. `lib/utils/timelineDates.ts` [NOUVEAU]
 - **Moteur de calcul de date unique et partagé :**
 - Centralisation de `computeMilestoneTargetDate` pour garantir une concordance temporelle mathématiquement absolue entre toutes les vues de l'application.
2. `app/(couple)/espace-couple/result/page.tsx`
 - Utilisation de la fonction unifiée `computeMilestoneTargetDate`.
3. `app/(couple)/espace-couple/planning/page.tsx`
 - **Synchronisation totale avec les jalons de la frise :**
 - Chaque tâche hérite directement de la date calculée pour son jalon parent sur la frise.
 - **Correction du décalage négatif anti-date passée :** Remplacement de l'étalement négatif qui avait fait glisser une tâche au 8 septembre par un étalement strictement positif vers l'avant (`+0, +2, +4 jours`), avec verrou absolu `≥ aujourd'hui`.
 - **Ergonomie à double niveau :**
 - **Clic direct sur une tâche :** Ouvre immédiatement la modale de modification de cette tâche précise.
 - **Clic sur la case du jour :** Ouvre la vue complète des étapes de la journée.

12. Évolution Implémentée : Choix d'une Date Précise dans le Formulaire « Nouvelle Étape » (Mode Hybride Date Picker / Échéance Relative)

📌 Contexte & Diagnostic

Auparavant, lors de la création d'une nouvelle étape, l'utilisateur ne pouvait renseigner qu'un mois relatif (« *X mois avant le jour J* »), le jour précis étant calculé automatiquement par l'application.

Dans l'organisation réelle d'un mariage, les couples ont souvent des rendez-vous et échéances très précises (ex : « *Dégustation traiteur le samedi 24 octobre* », « *Rendez-vous essayage robe le 18 novembre* »).

🔧 Solution Implémentée & Fonctionnalités Livrées

1. Mode Hybride Élégant dans la Modale Planning (`app/(couple)/espace-couple/planning/page.tsx`) :

- **Sélecteur de mode à onglets pilules discrets** : Intégration parfaite dans la charte graphique (`rounded-full` , fond `#fef2f4` , bordure `#EDEDf0` , typographie harmonieuse).
- **Mode « 📅 Date précise » (Recommandé / Par défaut)** :
 - Champ sélecteur de date natif stylisé (`rounded-[28px]` , bordure `border-2 border-[#EDEDf0]` , texte `#0E0E10`).
 - Bornage de sécurité : date minimale fixée à aujourd'hui (`min`), date maximale bornée au jour J du mariage (`max`).
 - Rétro-calcul automatique : calcul instantané et silencieux du nombre de mois relatifs correspondants pour conserver une synchronisation parfaite avec les filtres et les jalons.
- **Mode « ⏳ Mois avant le jour J »** :
 - Conservation intégrale du menu déroulant existant pour les couples qui n'ont pas encore arrêté de date fixe.

2. Support Backend & Base de Données (`types/marketplace.ts` & `app/api/couple/tasks/route.ts`) :

- Ajout du champ `dueDate?: string | null` sur l'interface `TimelineTask` .
- Prise en compte dans `TaskSchema` (création) et `UpdateSchema` (mise à jour) de l'API `/api/couple/tasks` .
- Persistance sécurisée via `taskRepo.create` et `taskRepo.update` .

3. Affichage Direct sur le Calendrier :

- La fonction `getTaskTargetDate()` applique en priorité absolue `task.dueDate` , positionnant automatiquement l'étape sur le jour exact choisi par les mariés.

13. Moteur de Planification IA avec Dates Précises par Tâche & Groupement d'Activités

✦ Contexte & Diagnostic du problème identifié

1. Pollution visuelle des badges unitaires (Résolu immédiatement) :

- Le calendrier affichait un badge rose « 1 étape » sur chaque case contenant une tâche, alourdissant considérablement la lecture.
- *Action immédiate appliquée* : Le badge ne s'affiche désormais que lorsqu'une journée regroupe **plusieurs étapes** (`dayTasks.length > 1`), indiquant clairement un regroupement d'actions.

2. Limite de l'étalement algorithmique frontend (+0, +2, +4 jours) :

- Auparavant, l'IA ne générait qu'un délai mensuel relatif (`monthsBeforeWedding`) au niveau du jalon et une liste de libellés de tâches en texte brut (`tasks: string[]`).
- Le code frontend devait alors deviner le jour du mois. Pour éviter que toutes les tâches ne tombent le même jour, un décalage artificiel de +2 jours était appliqué.
- **Conséquence néfaste** : Dans la vraie vie, un couple peut et doit souvent **réaliser plusieurs tâches le même jour** (ex : samedi de visites de lieux, dégustation traiteur + choix du gâteau l'après-midi, démarches en mairie un matin de semaine). L'étalement artificiel empêchait ce regroupement naturel.

🔧 Solution Implémentée & Livrables Techniques

L'Intelligence Artificielle génère désormais directement la date recommandée (`suggestedDate` / `dueDate`) pour chaque tâche selon des règles métier rigoureuses, avec regroupement intelligent d'activités sur la même journée lorsque cela est opportun.

1. Critères Métier appliqués par l'IA lors de la planification

• Différenciation Week-end vs Semaine ouvrée :

- **Samedis / Dimanches** (`dayContext: "weekend"`) : Réservés aux activités nécessitant des déplacements ou la présence de proches (visite de domaines, dégustation de menu, essayage de tenues, salon du mariage, EVJF/EVG).
- **Lundi au Vendredi** (`dayContext: "weekday"`) : Dédiés aux formalités administratives et contractuelles (dépôt du dossier en mairie, rendez-vous notaire, signatures de devis, virements bancaires d'acomptes).

• Groupement Intelligent d'activités (Batching) :

- L'IA assigne la même date exacte à 2 ou 3 tâches complémentaires pour optimiser les journées d'organisation des futurs mariés.

• Respect strict de la chronologie et des contraintes légales :

- Publication des bans : obligatoirement au moins 10 jours avant la date du mariage.
- Clôture des RSVP : 6 à 8 semaines avant le mariage.
- Essayages finaux : 3 à 4 semaines avant le jour J.

• Bornage temporel dynamique & Zéro date passée :

- L'IA reçoit la date du jour (`now`) et la date du mariage (`weddingDate`) pour garantir des dates réalistes et chronologiques.

2. Évolution du Schéma IA & Rétrocompatibilité Totale

- `types/domain.ts` & `types/marketplace.ts` :
 - Définition de l'interface `TimelineTaskItem` (`title` , `suggestedDate?` , `dayContext?` , `reasoning?`).
 - Typage tolérant `tasks: (string | TimelineTaskItem)[]` sur `TimelineMilestone` et présence de `dueDate?: string | null` sur `TimelineTask`.
- `lib/ai/schema.ts` :
 - Déclaration de `TimelineTaskItemSchema` et union `TaskItemOrStringSchema = z.union([z.string().min(1), TimelineTaskItemSchema])` assurant une compatibilité sans faille avec les réponses IA existantes ou structurées.
- `lib/ai/prompts/timeline.prompt.ts` :
 - Configuration du prompt système avec les directives de week-end vs jours ouvrés, regroupement (batching) et format `YYYY-MM-DD`.
 - Injection des dates précises du jour et du mariage dans le prompt utilisateur.
- `lib/db/repositories/taskRepo.ts` :
 - Mise à jour de `createFromTimeline` pour extraire automatiquement le libellé et la date d'échéance `dueDate` de chaque tâche structurée générée par l'IA.
- `app/(couple)/espace-couple/planning/page.tsx` :
 - Import initial intégrant le `dueDate` suggéré par l'IA.
 - Fonction `getTaskTargetDate()` appliquant immédiatement `task.dueDate` sans recalcul heuristique.
- **Vues de restitution du plan** :
 - `app/(couple)/espace-couple/result/page.tsx` et `TimelineSection.tsx` adaptés pour afficher proprement les libellés quelle que soit la structure (string ou objet).

14. Automatisation des Appels d'Offres : Pré-sélection du Type de Prestataire & Estimation Budgétaire

Problèmes identifiés

1. Perte du contexte de la catégorie sélectionnée :

- Lorsqu'un utilisateur cliquait sur un type de prestataire (ex : *Traiteur*, *Photographe* / *Vidéaste*) puis cliquait sur « *Lancer mon appel d'offres* », la modale s'ouvrait avec le menu déroulant sur « *Choisir une catégorie* » (vide). L'utilisateur devait chercher et re-sélectionner manuellement la catégorie.

2. Saisie manuelle fastidieuse du budget :

- Les champs « *Min* » et « *Max* » du budget étaient systématiquement vides, alors que l'application dispose déjà d'un budget global pour le mariage et d'une ventilation financière issue du plan IA.

3. Condition de désactivation bloquante (`!formTouched`) :

- Le bouton d'envoi exigeait formellement que le formulaire ait été touché manuellement, ce qui empêchait la validation directe d'une demande dont les informations étaient déjà complètes.

Fichiers créés & modifiés

1. `lib/utils/budgetEstimator.ts` [CRÉATION]

- **Moteur d'estimation budgétaire contextuel** (`estimateBudgetForCategory`) :
 - **Priorité 1 (Plan IA)** : Récupère les fourchettes précises (`realisticMin` et `realisticMax` ou `planned`) générées pour le poste concerné dans l'audit IA.
 - **Priorité 2 (Budget global du couple)** : Si aucun plan IA n'est encore généré mais que le couple a défini un budget global et sélectionné ce métier dans ses catégories souhaitées, applique les ratios certifiés du marché français sur le montant total.
 - **Règle stricte d'absence de budget (Zéro valeur par défaut arbitraire)** : S'il n'y a aucun budget disponible ou alloué à cette catégorie (ou si aucun budget n'a été défini), le système ne pré-remplit rien. Les champs Min et Max restent rigoureusement vides (`""`) pour laisser l'utilisateur libre de sa saisie.
 - Arrondi propre des montants à la tranche de 50 € la plus proche.

2. `components/couple/TenderFormModal.tsx`

- **Synchronisation dynamique à l'ouverture** :
 - Dès l'ouverture de la modale avec une catégorie pré-sélectionnée, le menu déroulant est automatiquement calé sur cette catégorie (`value={category}`).
 - La tranche de budget (`budgetMin` et `budgetMax`) est instantanément renseignée avec les valeurs estimées.
- **Réactivité au changement de sélection** :
 - Si l'utilisateur change de catégorie dans le menu déroulant, les tranches budgétaires s'ajustent immédiatement au nouveau métier choisi.
- **Indicateur visuel rassurant** :
 - Ajout d'un badge «  Auto-rempli » et d'un message contextuel informant le couple de la source de l'estimation (« *Estimation issue de votre plan IA personnalisé* » ou « *Estimation calculée sur votre budget global* »).
 - Si l'utilisateur ajuste manuellement les montants, le badge disparaît naturellement pour respecter sa saisie.
- **Déblocage immédiat de l'action** :
 - Remplacement de la condition restrictive par `disabled={!launching || !category}` , permettant de lancer la demande en 1 clic.

3. `app/(couple)/espace-couple/prestataires/page.tsx`

- Chargement du plan IA (`/api/couple/result`) en parallèle des appels d'offres et transmission directe à `TenderFormModal` .
- Préservation de la catégorie active lors du passage par la boîte de dialogue de confirmation de remplacement des suggestions.

4. `app/(couple)/espace-couple/prestataires/nouveau/page.tsx`

- Intégration du moteur d'estimation budgétaire automatique sur la page dédiée `/espace-couple/prestataires/nouveau`.
- Support de la pré-sélection par URL (`?category=...`).

15. Enrichissement Automatique des Exigences Spécifiques & Priorités depuis le Quiz

Problème identifié

Lors de la création d'un appel d'offres (dans `TenderFormModal` ou sur `/prestataires/nouveau`), les champs « **Exigences spécifiques** » et « **Priorité principale** » restaient systématiquement vides. L'utilisateur devait retaper à la main des informations qu'il avait déjà renseignées lors de son parcours initial (notamment à l'**Étape 7 : Besoins spécifiques des invités** et à l'**Étape 9 : Priorité n°1**).

Fichiers modifiés & Détails des changements

1. `lib/utils/budgetEstimator.ts`

- **Fonction** `estimateRequirementsForCategory(category, project)` :
 - **Règle stricte d'authenticité (Zéro extrapolation)** : Seules les exigences expressément et personnellement renseignées par le couple à l'**Étape 7 du Quiz (Besoins spécifiques)** sont pré-remplies.
 - **Traiteur** : Traduction exacte des régimes alimentaires cochés (*Végétarien, Vegan, Halal, Casher, Sans gluten*) et retranscription textuelle intégrale du champ libre des allergies réelles saisies au clavier par le couple (*Allergies : ...*).
 - **Lieu de réception** : Restitution de l'accès PMR (*Accès PMR - personnes à mobilité réduite*) uniquement si le couple a explicitement coché « *Oui* » à la question sur la mobilité réduite.
 - **Toutes les autres catégories (Transport, Photographe, DJ, Fleuriste, Déco, etc.)** : Aucun besoin spécifique n'ayant été demandé ou configuré pour ces métiers lors du Quiz, **le champ reste rigoureusement vide**. Aucune extrapolation artificielle (pas de navettes gare/aéroport inventées, pas de duplication des styles/ambiances).
- **Fonction** `estimatePriorityForCategory(category, project)` :
 - Restitution claire de la priorité n°1 déclarée par le couple au Quiz (*Maîtriser le budget, Lieu coup de cœur, Qualité de prestation, Confort des invités, Zéro stress*). Reste vide si non renseignée.

2. `components/couple/TenderFormModal.tsx`

- Pré-remplissage automatique des champs dès l'ouverture de la modale.
- Ré-évaluation dynamique en temps réel si l'utilisateur change de corps de métier dans la liste déroulante.
- Ajout d'un badge distinctif mauve «  *Issu de vos réponses Quiz* » et «  *Priorité n°1 Quiz* » au-dessus des champs pour valoriser l'intelligence de la plateforme.

- Préservation de la liberté totale de modification ou d'ajout par l'utilisateur.

3. `app/(couple)/espace-couple/prestataires/nouveau/page.tsx`

- Application de la même logique d'auto-remplissage et d'indicateurs visuels sur la page autonome `/espace-couple/prestataires/nouveau`.

16. Synchronisation & Restitution Intégrale du Profil Prestataire (Site Web, Téléphone, Email & Validation)

Problèmes identifiés

1. **Site web absent du portfolio après inscription** : Lors de l'inscription sur `/devenir-professionnel`, l'URL saisie dans le champ `form.website` était transmise à la racine du payload mais `portfolio.website` recevait une variable interne vide `form.portfolioWebsite`. Sur `/espace-prestataire/portfolio`, le composant ne lisant que `portfolio.website`, le champ apparaissait vierge et le bouton Globe restait désactivé (`pointer-events-none`).
2. **Statut « Vérification en cours » persistant malgré validation admin** : Lorsque l'administrateur approuvait un profil via `/admin/candidatures`, l'API (`/api/admin/vendor/[id]`) mettait à jour le champ `status: "approved"`, mais laissait le booléen `verified` à sa valeur par défaut `false`. De plus, l'API publique `/api/public/vendors/[id]` omettait la propriété `verified`, masquant le sceau de certification sur la fiche.
3. **Coordonnées masquées (Téléphone & Email)** : L'API `/api/public/vendors/[id]` filtrait et omettait délibérément `phone` et `email` dans `pickPublicVendor`. Par conséquent, sur la prévisualisation du portfolio (`/prestataires/preview/[vendorId]`), les coordonnées affichaient systématiquement « Non renseigné » malgré leur enregistrement en base de données.

Fichiers modifiés & Détails des changements

1. `app/(marketing)/devenir-professionnel/page.tsx`

- Synchronisation lors de l'envoi de la candidature : `portfolio.website` est désormais alimenté par `form.website || form.portfolioWebsite || null`.

2. `app/api/vendor/apply/route.ts`

- Harmonisation bidirectionnelle : Le profil créé stocke l'URL du site web à la fois à la racine (`website`) et dans l'objet imbriqué (`portfolio.website`).

3. `app/(vendor)/espace-prestataire/portfolio/page.tsx`

- Lecture résiliente : `setWebsite(json.profile?.portfolio?.website || json.profile?.website || "")`.
- Enregistrement synchronisé : Lors de la sauvegarde du portfolio, le site web est mis à jour à la fois à la racine et dans l'objet `portfolio`.

- Bouton Globe sécurisé : Préfixage automatique en `https://` pour éviter toute redirection relative si le prestataire a omis le protocole.
4. `app/(vendor)/espace-prestataire/profil/page.tsx`
 - Lors de la modification des coordonnées dans l'espace profil, la mise à jour du champ `website` est automatiquement propagée à `portfolio.website`.
 5. `app/api/admin/vendor/[id]/route.ts`
 - Activation automatique du label vérifié : Lorsque l'admin passe un statut à `"approved"`, le profil prestataire reçoit désormais automatiquement `verified: true` (et `verified: false` si rejeté). Déclenche les +5 points d'expérience et de réputation dans le moteur de matching IA.
 6. `app/api/public/vendors/[id]/route.ts`
 - Restitution complète des données publiques autorisées : Ajout de `phone`, `email`, `verified` et `status` dans l'objet retourné aux visiteurs et futurs mariés.
 7. `app/(marketing)/prestataires/preview/[vendorId]/page.tsx` & `app/(couple)/espace-couple/prestataires/profil/[vendorId]/page.tsx`
 - Restitution des coordonnées cliquables : Le téléphone génère un lien d'appel direct `tel:...` et l'email un lien `mailto:...`.
 - Tampon et statut de vérification : Affichage effectif du sceau « Profil vérifié » et de la mention « Vérification : Vérifié » en vert dès que le profil est approuvé.

17. Importation Sécurisée des Avis Google Business Profile avec Verrou Anti-Usurpation

Contexte & Objectif

Permettre aux professionnels inscrits sur la plateforme d'importer leurs avis clients réels depuis leur fiche Google (Google Business Profile / Google Maps) pour renforcer la crédibilité de leur portfolio et accélérer la signature avec les futurs mariés.

Exigence absolue de sécurité : Garantir de façon infaillible que seul le véritable propriétaire de l'établissement puisse importer les avis de sa fiche, éliminant tout risque d'usurpation où un prestataire malveillant s'approprierait la réputation d'un confrère concurrent.

Architecture & Verrous Techniques Implémentés

1. Verrou Anti-Usurpation & Service Dédié (`lib/services/googleBusinessService.ts`)

- **Normalisation & Concordance stricte de domaine** (`normalizeDomain`, `checkDomainMatch`) :

- Extraction automatique du nom de domaine officiel renseigné sur la fiche Google Maps (ex. `chateau-saint-martin.com`).
- Comparaison avec le site web déclaré ou l'adresse email professionnelle du prestataire.
- **Défi de sécurité OTP (6 chiffres, TTL 15 minutes) :**
 - Transmission d'un code de vérification unique à l'adresse email du domaine (`contact@chateau-saint-martin.com`) ou à l'adresse de compte.
 - Masquage de sécurité de l'adresse de destination (ex. `co***@chateau-saint-martin.com`).
- **Support Google Places API & Sandbox haute fidélité :**
 - Intégration directe avec l'API Google Places (`GOOGLE_PLACES_API_KEY`) pour la recherche d'établissement, les avis réels et le Place ID.
 - Mode sandbox certifié haute fidélité pour le développement local et les démonstrations.

2. Routes API Dédiées (`app/api/vendor/google-business/...`)

- `POST /lookup` : Recherche instantanée de l'établissement par nom ou URL Maps avec contexte prestataire.
- `POST /request-verification` : Analyse de concordance anti-usurpation et émission du défi OTP.
- `POST /verify` : Validation du code OTP, certification de la fiche et importation automatique des avis dans `vendor.portfolio.googleBusiness` .
- `POST /sync` : Synchronisation en 1 clic des derniers avis et de la note Google.
- `POST /disconnect` : Dissociation sécurisée de la fiche à la demande du professionnel.

3. Espace Prestataire (`app/(vendor)/espace-prestataire/portfolio/page.tsx`)

- **Intégration directe & intuitive dans la card « Avis clients »** : Bouton officiel Google placé à côté du bouton d'ajout d'avis manuel (`+`) pour éviter la redondance d'une carte séparée.
- **Modale épurée en 3 étapes** : Présentation claire et sans verbiage anxiogène (Recherche de l'établissement > Confirmation de la fiche > Code de validation).
- **Affichage combiné des avis** : Encart certifié Google avec lien vers Google Maps, bouton de resynchronisation et affichage distinctif des avis officiels importés et des avis manuels.

4. Restitution Côté Mariés & Portfolio Public

- **Portfolio Public** (`app/(marketing)/prestataires/preview/[vendorId]/page.tsx`) :
 - Badge distinctif officiel Google dans la fiche technique latérale : « *Avis Google certifiés ★★★★★ (4.9/5 - 42 avis)* » avec lien de vérification direct vers Google Maps.
 - Onglet « *Avis & Réputation* » : Mise en valeur prioritaire des avis certifiés Google avec logo officiel Google, note et date.
- **Espace Couple** (`app/(couple)/espace-couple/prestataires/profil/[vendorId]/page.tsx`) :
 - Intégration miroir du badge certifié Google et des avis certifiés pour les futurs mariés consultant le profil du prestataire.

5. Moteur de Matching IA (`lib/matching/engine.ts`)

- **Bonus de réputation (+5 points)** dans le calcul algorithmique de compatibilité (`computeRuleBasedMatchScore`) pour tout prestataire dont la fiche Google est certifiée avec une note `≥ 4.0/5` .

- Transmission des données certifiées Google au modèle d'analyse IA (`googleCertified` dans le prompt).

18. Correction & Support Universel des Vidéos Portfolio (YouTube, Shorts, Vimeo, MP4)

Problème Identifié

- Les prestataires enregistraient des liens vers des vidéos de présentation au format **YouTube** ou **YouTube Shorts** (ex. `https://youtube.com/shorts/U3E3TT3S1uQ...`).
- Côté front-end, l'application utilisait une balise native `<video src={url} controls />` :
 - **Incompatibilité technique** : Un élément HTML5 `<video>` est exclusivement conçu pour des fichiers conteneurs bruts (`.mp4` , `.webm` , `.ogg`). Il est incapable de lire une page Web YouTube ou Vimeo, générant un écran noir ou une erreur de décodage média.
 - De plus, les vidéos n'étaient pas intégrées sur la page de prévisualisation publique (`/prestataires/preview/[id]`), et la gestion dans l'espace prestataire n'affichait aucun retour visuel.
 - La mention `Dossier n° [HASH]` affichait un code aléatoire de base de données donnant une impression de texte technique non nettoyé.

Correctifs & Améliorations Apportés

1. **Création du composant unifié** `VideoEmbed` (`components/shared/VideoEmbed.tsx`) :
 - Détection automatique et conversion à la volée des URLs :
 - **YouTube Shorts** (`youtube.com/shorts/...`) → lecteur iframe embarqué `youtube-nocookie.com/embed/...`.
 - **YouTube classique** (`watch?v=...` , `youtu.be/...` , `embed/...`).
 - **Vimeo** (`vimeo.com/...`) → lecteur `player.vimeo.com/video/...`.
 - **Dailymotion** (`dailymotion.com/video/...` , `dai.ly/...`).
 - **Fichiers vidéo directs** (`.mp4` , `.webm` , Cloudinary, S3).
 - Rendu fluide, responsive (`aspect-video`), avec paramètres de confidentialité et contrôle d'accès sécurisé.
2. **Intégration sur la fiche profil couple** (`/espace-couple/prestataires/profil/[vendorId]`) :
 - Remplacement de la balise `<video>` obsolète par le composant `VideoEmbed`.
 - Affichage immédiat et fluide des vidéos et Shorts des prestataires.
 - Remplacement de la mention `Dossier n°` par un format élégant `Réf. #...`.
3. **Intégration sur la prévisualisation publique** (`/prestataires/preview/[vendorId]`) :

- Ajout de la section dédiée « *Vidéos de présentation* » qui était manquante.
- Mise à niveau de l'en-tête de référence en `Réf. #...`.

4. Amélioration de l'espace prestataire (`/espace-prestataire/portfolio`) :

- Prévisualisation interactive des vidéos ajoutées directement dans le tableau de bord avec lecteur embarqué.
- Précision d'aide contextuelle informant le professionnel des formats pris en charge (YouTube, Shorts, Vimeo, MP4).
- Suppression fluide en un clic.

19. Proposition d'Évolution : Affichage du Budget Dédié par Prestation et Saisie du Tarif Chiffré dans la Réponse à l'Appel d'Offres

Contexte & Constat

- **Problème d'ambiguïté budgétaire** : Dans la modale de réponse à un appel d'offres (`/espace-prestataire/appels-offres`), la ligne budgétaire affiche actuellement le budget global de tout le projet de mariage (ex. « *Budget : 20 000 EUR* »).
- **Impact côté prestataire** : Pour un corps de métier spécifique (ex. *Lieu de réception* ou *Photographe*), voir 20 000 € est trompeur car le professionnel peut supposer que cette somme lui est intégralement destinée, alors que le couple a fixé une enveloppe ciblée pour ce besoin (ex. 1 000 € - 9 000 €), le reste finançant le traiteur, les tenues, la décoration, etc.
- **Absence de champ chiffré dans la réponse** : Actuellement, le formulaire de réponse ne propose qu'une zone de texte libre (« *Votre message* »). Le prestataire doit écrire son prix au milieu de son message, ce qui complique la comparaison rapide des devis par les futurs mariés.

Évolutions & Améliorations Proposées

1. Affichage Transparent & Double Niveau du Budget

- **Niveau 1 (Prioritaire)** : Afficher l'enveloppe budgétaire allouée spécifiquement à la prestation :

Budget pour cette prestation ([Catégorie]) : [Min] € – [Max] € (ex. 1 000 € - 9 000 €)

- **Niveau 2 (Informatif)** : Préciser le budget global pour permettre au prestataire d'évaluer le standing de l'événement :

(Budget global du mariage : 20 000 €)

- **Fallback** : Si le couple n'a pas défini de fourchette précise, indiquer « *Selon devis / proposition* » avec l'estimation suggérée par le planificateur IA.

2. Intégration d'un Champ « Tarif Proposé (€) » dans la Modale

- Ajouter un champ numérique dédié : « **Votre tarif estimatif (€)** » (*optionnel ou recommandé*).
- Exploiter le champ `amount` déjà prévu et validé dans le schéma backend `ProposalSchema` (`app/api/vendor/proposals/route.ts`).
- **Affichage sur l'espace couple** : La carte de candidature reçue affichera en gras le montant proposé (ex. « *Proposition : 2 500 €* »), permettant une comparaison instantanée et transparente entre les différents prestataires candidats.

20. Correctifs du Workflow des Appels d'Offres : Résolution « Projet introuvable », Statuts Dynamiques (« Répondu », « Retenu / Validé », « Clôturé ») et Célébration

Problèmes Identifiés & Analysés

1. Erreur « Projet introuvable » lors du clic prestataire :

- *Cause racine* : Sur la page `/espace-prestataire/propositions`, le bouton redirigeait vers `/espace-prestataire/appels-offres/${proposal.tenderId}` (l'identifiant de l'appel d'offres `tenderId`), tandis que l'API attendait impérativement un `matchId`. L'API renvoyait une erreur 404.

2. Bouton « Répondre » persistant même après acceptation par les mariés :

- *Cause racine* : Le code vérifiait uniquement `match.status === "contacted"`. Dès que le couple validait/sélectionnait le prestataire, `match.status` passait à `"accepted"`, faisant échouer la condition et affichant de nouveau par erreur le bouton rouge « Répondre » !

3. Manque de feedback visuel clair sur la sélection :

- Le prestataire retenu par les mariés n'avait aucun moyen immédiat de savoir qu'il avait remporté l'appel d'offres, et la possibilité d'ignorer ou de re-répondre créait de la confusion.

Correctifs Apportés & Fonctionnalités Implémentées

1. API Résiliente avec Fallback Universel (`app/api/vendor/opportunities/[id]/route.ts`) :

- Supporte désormais aussi bien un `matchId`, un `tenderId` ou un `projectId`.
- Si la recherche directe par clé échoue, une recherche par correspondance (`m.tenderId === id || m.projectId === id`) retrouve instantanément le bon match et les détails complets du projet.
- L'objet `tender` avec ses fourchettes budgétaires ciblées est désormais retourné systématiquement.

2. Affichage Dynamique et Cohérent des Statuts sur `/espace-prestataire/appels-offres` :

- **Avant réponse** : Bouton rouge distinctif « **Répondre** » (ou « Activer » si offre gratuite).
 - **Après réponse** : Badge vert doux « ✓ **Répondu** » (`bg-[#e4f4ed] text-[#2e7d5e]`), bloquant toute double soumission accidentelle.
 - **Prestataire sélectionné/validé par le couple** : Badge d'honneur mis en valeur « **Retenu / Validé** 🎉 » (`bg-[#d8ecd9] text-[#2e7d5e] font-bold border border-[#2e7d5e]/20`).
 - **Appel d'offres attribué à un autre prestataire ou clôturé** : Badge neutre « **Clôturé** » (`bg-[#EDEDf0] text-[#6B6B72]`).
 - **DossierCard & Table Desktop** : Le bouton de suppression/ignorance (X) est automatiquement masqué dès qu'un prestataire a été validé pour son mariage.
3. **Page Détail du Projet** (`/espace-prestataire/appels-offres/[matchId]`) :
- **Bannière de félicitations** 🎉 : Affichage d'un encart de célébration haut de gamme informant le prestataire qu'il a été choisi par les mariés avec un bouton d'accès direct à la messagerie (`/espace-prestataire/messagerie`).
 - **Fiche technique & version mobile** : Statut « *Retenu pour ce mariage* 🎉 » ou « *Proposition envoyée* » sur desktop et mobile.

21. Proposition d'Évolution : Synchronisation Collaborative du Calendrier et des Tâches Préparatoires entre le Couple et le Prestataire Retenu

🎯 Contexte & Constat

- **Synchronisation actuelle partielle** : Dès qu'un couple retient et valide un prestataire (`match.status === "accepted"`), seule la date du jour J (`project.weddingDate`) est injectée dans le calendrier du professionnel (`/espace-prestataire/calendrier`).
- **Besoin opérationnel fort** : Dans l'organisation réelle d'un mariage, la prestation ne se limite pas au jour du mariage. Elle implique des étapes préparatoires cruciales et des jalons intermédiaires datés :
 - *Lieu de réception* : Visite technique du domaine, dépôt du chèque de caution, état des lieux d'entrée et remise des clés.
 - *Traiteur* : Dégustation test des menus, validation des régimes/allergies, décompte final des convives à M-1.
 - *Photographe / Vidéaste* : Séance engagement / shooting préliminaire, brief des photos de groupe et repérage lumière.
 - *DJ / Orchestre* : Sélection de la playlist / blacklist, calage des animations avec les témoins, répétition technique.
- **Cloisonnement actuel** : Ces jalons et checklists restent cantonnés sur l'espace planning du couple (`/espace-couple/planning`), obligeant le couple et le prestataire à s'échanger des e-mails, SMS ou appels externes pour synchroniser ces dates clés.

1. Filtrage et Rapprochement Automatique par Catégorie Métier

- L'API du calendrier prestataire (`app/api/vendor/calendar/route.ts`) interroge les jalons IA et les tâches personnalisées du projet mariage (`timeline.milestones` & `timeline_tasks`).
- Filtrer les tâches et sous-étapes dont les mots-clés ou l'étiquette correspondent au corps de métier du prestataire (`match.category`).
- Calcul automatique de la date cible selon la formule `weddingDate - monthsBeforeWedding` (ou date butoir fixée par le couple `dueDate`).

2. Affichage Visuel Hiérarchisé sur le Calendrier Prestataire

- **Jour J (Le Mariage)** : Badge plein rouge corail (`#e64a5d`) mis en avant comme l'événement principal.
- **Jalons Préparatoires & RDV** : Badges élégants aux tons doux (lavande ou ambre) indiquant clairement la nature du jalon (ex. « *Dégustation menu* », « *Visite technique* »).
- **Fiche détail au clic** : En cliquant sur un jalon, le prestataire peut consulter les consignes du couple, l'échéance et les notes associées.

3. Tâches Collaboratives et Statut Partagé

- Possibilité pour le prestataire ou le couple de cocher un jalon préparatoire comme « Réalisé » (ex. « *Acompte versé* », « *Choix du menu validé* »).
- Notification automatique à l'autre partie dès qu'un jalon d'étape est franchi.

22. Système d'Indicateurs et Pastilles de Notification Visuelles sur les Menus de Navigation (Espace Prestataire, Admin et Couple)

🎯 Contexte & Objectif

Afin que les utilisateurs soient informés instantanément des actions requises sans devoir parcourir manuellement chaque onglet :

- **Espace Prestataire** : Alertes visuelles pour les nouveaux messages non lus et les nouvelles opportunités d'appels d'offres ciblées.
- **Espace Administrateur** : Alertes visuelles pour les dossiers de candidatures de prestataires en attente de vérification et les tickets de support ouverts.
- **Espace Couple** : Alertes visuelles pour les messages des prestataires et les devis / candidatures reçus.

[!IMPORTANT] **Respect Intégral du Design Existant** : Aucune mise en page n'a été déformée. Les badges adoptent un format micro-pill élégant ou pastille discrète parfaitement harmonisée avec la charte graphique de

chaque espace (rouge corail `#E05A47` / `#e64a5d` pour le couple et le prestataire, rose vif `#db2777` pour l'espace admin).

Architecture Technique & Implémentation

1. Endpoints API Dédiés et Légers

Trois endpoints d'interrogation rapide et sécurisés ont été créés :

- `app/api/vendor/badge-counts/route.ts` :
 - `unreadMessages` : Décompte des messages où `unread === true` et `senderRole !== "vendor"`.
 - `newOpportunities` : Décompte des appels d'offres matchés ayant le statut `"suggested"`.
 - `unreadNotifications` : Décompte des notifications non lues.
- `app/api/admin/badge-counts/route.ts` :
 - `pendingCandidatures` : Nombre de prestataires inscrits ayant le statut `"pending"`.
 - `openTickets` : Nombre de tickets de support actifs (`status === "open"`).
- `app/api/couple/badge-counts/route.ts` :
 - `unreadMessages` : Décompte des messages où `unread === true` et `senderRole !== "couple"`.
 - `pendingProposals` : Décompte des propositions soumises par des prestataires en statut `"pending"`.

2. Affichage Réactif et Non-Intrusif sur les Menus

- **Espace Prestataire** (`app/(vendor)/espace-prestataire/VendorLayoutClient.tsx`) :
 - Navigation Desktop principale : Pastille sur « **Messagerie** » et « **Appels d'offres** ».
 - Menu secondaire déroulant : Pastilles préservées sur les liens correspondants.
 - Header mobile : Pastille de notification sur le bouton hamburger dès qu'une action est requise.
 - Drawer mobile : Pastilles chiffrées sur chaque ligne de menu.
 - Bottom Tab Bar mobile : Pastille rouge de notification positionnée avec précision sur les icônes de la barre d'onglets inférieure.
- **Espace Administrateur** (`app/admin/AdminLayoutClient.tsx`) :
 - Sidebar Desktop & Mobile Drawer : Badge chiffré rose (`#db2777`) sur l'onglet « **Candidatures** » (ex: nombre de dossiers à valider) et sur l'onglet « **Support** ».
- **Espace Couple** (`app/(couple)/espace-couple/CoupleLayoutClient.tsx`) :
 - Navigation Desktop & Mobile : Pastille sur « **Prestataires** » et « **Messagerie** ».
 - Bouton mobile « Plus » et Bottom Tabs dotés d'indicateurs visuels.

3. Rafraîchissement Automatique & Temps Réel

- **Revalidation périodique** : Polling toutes les 30 secondes en arrière-plan sans impact sur les performances.
- **Revalidation au focus** : L'état s'actualise dès que l'utilisateur revient sur l'onglet du navigateur (`window.addEventListener("focus")`).

- **Synchronisation immédiate** : Dès qu'une conversation est consultée sur la messagerie prestataire ou couple, un événement personnalisé `badge-counts-updated` réinitialise instantanément le badge sans délai.

4. Règle Stricte d'Extinction des Badges (Candidatures Validées ou Appels d'Offres Clôturés / Attribués)

Conformément à la consigne, aucun badge ou pastille ne doit subsister lorsqu'une candidature ou un appel d'offre n'est plus en attente d'action :

- **Côté Prestataire** (`/api/vendor/badge-counts`) :
 - Si le prestataire a déjà répondu à l'appel d'offres : exclusion immédiate (`newOpportunities` décrémenté).
 - Si le match a été validé ou accepté (`status === "accepted"`) : exclusion immédiate.
 - Si l'appel d'offres a été clôturé (`tender.status === "closed"`) ou validé/attribué à un prestataire (`tender.selectedProposalId`) : exclusion systématique même si le match brut était resté au statut de suggestion.
- **Côté Couple** (`/api/couple/badge-counts`) :
 - Si une candidature a été validée (`proposal.status === "accepted"`) : exclusion immédiate.
 - Si l'appel d'offres de la catégorie a été clôturé ou validé (`tender.status === "closed"` ou `selectedProposalId` défini) : toutes les autres propositions associées ne déclenchent plus aucun badge.
 - Dans la grille des prestataires (`espace-couple/prestataires`), le badge chiffré des recommandations est automatiquement ramené à zéro dès lors que le poste est validé ou clôturé.
- **Côté Administrateur** (`/api/admin/badge-counts`) :
 - Dès qu'un dossier de candidature est validé (*Approuvé*) ou refusé, l'événement `badge-counts-updated` éteint immédiatement le badge sur le menu « Candidatures ».